TP - Sécuriser un Serveur Web pour Héberger une Application Symfony VIA SSH

Ce TP couvre la sécurisation complète d'un serveur Debian destiné à héberger une application Symfony. Chaque étape apporte une couche de protection supplémentaire : mises à jour automatiques, compte non-root, authentification par clé SSH, restrictions du daemon SSH, pare-feu UFW, serveur Apache2, environnement PHP/Symfony et interface d'administration Cockpit.

Etape 1 - Mise à jour du système

Avoir un système à jour et réduire les failles de sécurité exploitées.

La première mesure de sécurité est de maintenir le système à jour. apt update récupère la liste des paquets disponibles, dist-upgrade installe les mises à jour en gérant les dépendances, et unattended-upgrades automatise l'application des correctifs de sécurité futurs sans intervention manuelle. La commande dpkg-reconfigure permet de choisir si les mises à jour de sécurité doivent être appliquées automatiquement.

Procédure

sudo apt update

sudo apt dist-upgrade -y

sudo apt install unattended-upgrades -y

sudo dpkg-reconfigure --priority=low unattended-upgrades

Etape 2 - Création d'un utilisateur limité

Éviter d'utiliser le compte root afin de réduire les risques en cas d'intrusion.

Travailler en root expose l'ensemble du système en cas de compromission. La commande adduser crée un compte standard (monuser) avec son répertoire home. usermod -aG sudo lui accorde les droits d'administration via sudo uniquement lorsque nécessaire. Ainsi, même si le compte est compromis, l'attaquant ne dispose pas directement de tous les privilèges root.

Procédure

sudo adduser monuser

sudo usermod -aG sudo monuser

Etape 3 - Clés SSH

L'authentification par clé publique/privée est bien plus sécurisée qu'un mot de passe. Une paire de clés RSA 4096 bits est générée sur la machine locale (ssh-keygen -b 4096). La clé privée reste localement et ne quitte jamais la machine. La clé publique est déposée sur le serveur dans ~/.ssh/authorized_keys. Le répertoire ~/.ssh doit avoir les permissions 700 (lisible uniquement par le propriétaire) pour que SSH l'accepte. Le transfert se fait via scp (Windows) ou ssh-copy-id (Linux/macOS).

Supprimer les mots de passe afin d'obliger une authentification par clé publique/privée.

➤ **Sur le serveur**

mkdir ~/.ssh && chmod 700 ~/.ssh

Installer le server SSH

sudo apt install openssh-server

➤ **Sur la machine locale**

ssh-keygen -b 4096

➤ **Transfert de la clé**

Windows :

scp $env:USERPROFILE/.ssh/id_rsa.pub monuser@IP:~/.ssh/authorized_keys

Linux/macOS :

ssh-copy-id monuser@IP

Etape 4 - Durcissement SSH

Éviter les attaques bruteforce classiques et empêcher l'accès root.

Fichier /etc/ssh/sshd_config

Port 8181

AddressFamily inet

PermitRootLogin no

PasswordAuthentication no

PermitEmptyPasswords no

Puis :

sudo systemctl restart sshd

Etape 5 - Pare-feu UFW

Limiter le trafic aux ports nécessaires.

UFW (Uncomplicated Firewall) est la surcouche simplifiée d'iptables sous Debian/Ubuntu. La politique par défaut bloque tout le trafic entrant (deny incoming) et autorise tout le trafic sortant. On n'ouvre que les ports strictement nécessaires : 8181/tcp pour SSH (nouveau port), 80/tcp pour HTTP et 443/tcp pour HTTPS. Le blocage du ping (ICMP echo-request) dans /etc/ufw/before.rules réduit la visibilité du serveur sur le réseau (reconnaissance plus difficile). sudo ufw enable active le pare-feu, sudo ufw reload applique les règles modifiées.

Installation & configuration

sudo apt install ufw -y

sudo ufw default deny incoming

sudo ufw default allow outgoing

sudo ufw allow 8181/tcp

sudo ufw allow 80/tcp

sudo ufw allow 443/tcp

sudo ufw enable

sudo ufw reload

Bloquer le ping

Modifier /etc/ufw/before.rules :

-A ufw-before-input -p icmp --icmp-type echo-request -j DROP

Appliquer les changements avec :

sudo ufw reload

Etape 6 - Installation d'Apache2

Installer le serveur web nécessaire pour recevoir le projet Symfony.

Apache2 est le serveur HTTP qui recevra les requêtes web et les redirigera vers l'application Symfony. apt install apache2 installe le paquet. systemctl enable apache2 le démarre automatiquement au boot. systemctl start apache2 le lance immédiatement. Apache doit être actif avant de configurer le VirtualHost Symfony, et les ports 80 et 443 doivent être ouverts dans UFW (étape 5) pour que le trafic web arrive jusqu'à lui.

Installation

sudo apt install apache2 -y

sudo systemctl enable apache2

sudo systemctl start apache2

Etape 7 - Préparer l'environnement Symfony + VirtualHost

Configurer le serveur pour exécuter un projet Symfony.

Symfony est un framework PHP moderne. Son déploiement nécessite PHP et ses extensions (xml, curl, mbstring, intl, mysql, opcache...), un VirtualHost Apache pointant vers le répertoire public/ du projet, et le module mod_rewrite pour le routage. Le VirtualHost indique à Apache comment servir le

site : ServerName définit le domaine, DocumentRoot pointe vers public/ (le seul dossier publiquement accessible), AllowOverride All permet à Symfony d'utiliser son fichier .htaccess pour le routage. Le déploiement via git clone récupère le code source, composer install télécharge les dépendances PHP sans les paquets de développement (--no-dev). Les permissions sont ensuite ajustées pour que www-data (l'utilisateur Apache) puisse écrire dans le répertoire var/.

---

1. Installation de PHP et extensions

sudo apt install php php-cli php-fpm php-xml php-curl php-mbstring php-zip php-intl php-mysql php-opcache -y

---

2. Création d'un VirtualHost

sudo nano /etc/apache2/sites-available/symfony.conf

Contenu :

```
<VirtualHost *:80>
   ServerName monsite.local
   DocumentRoot /var/www/symfony/public

   <Directory /var/www/symfony/public>
      AllowOverride All
      Require all granted
   </Directory>
</VirtualHost>
```

Activation :

sudo a2ensite symfony.conf

sudo a2enmod rewrite

sudo systemctl reload apache2

---

3. Déploiement de Symfony

cd /var/www

sudo git clone <https://github.com/Daniween/CYBER_SYMFONY_1> symfony

cd symfony

composer install --no-dev --optimize-autoloader

Permissions :

sudo chown -R www-data:www-data /var/www/symfony

sudo chmod -R 775 /var/www/symfony/var

Etape 8 - Outils d'administration

Cockpit est une interface d'administration web pour les systèmes Linux. Elle permet de surveiller les ressources (CPU, RAM, disque, réseau), de gérer les services systemd, les journaux système et les mises à jour depuis un navigateur web. Accessible sur le port 9090 (https://IP:9090), elle offre une alternative graphique aux commandes SSH. L'ouverture du port 9090 dans UFW est nécessaire pour y accéder depuis une machine distante. systemctl enable cockpit.socket assure son démarrage automatique au boot.

Cockpit

sudo apt install cockpit -y

sudo systemctl enable cockpit.socket

sudo ufw allow 9090/tcp

Conclusion

Ce TP a mis en place une chaîne complète de sécurisation d'un serveur web Debian hébergeant une application Symfony. Les mesures appliquées couvrent plusieurs niveaux : système (mises à jour automatiques), accès (compte non-root + sudo), réseau (SSH sur port non standard + pare-feu UFW strict), authentification (clés SSH, root interdit, mot de passe désactivé), et applicatif (Apache2, PHP, Symfony déployé avec les bonnes permissions). Chaque étape renforce la sécurité globale selon le principe de défense en profondeur : la compromission d'une couche ne doit pas entraîner la compromission du système entier.